

- The **final keyword** in java is used to restrict the user.
- The java final keyword can be used in many context.
 1. variable
 2. method
 3. class

Final Variable → To create constant variables

Final Methods → Prevent Method Overriding

Final Classes → Prevent Inheritance

final variable

- A variable can be declared as final.
- Doing so **prevents** its contents from being modified.
- This means that you must initialize a final variable when it is declared.
- It is a common coding convention to choose all uppercase identifiers for **final** variables.
- Variables declared as **final** **do not occupy** memory on a per-instance basis.
- Thus, a final variable is essentially a constant.
- A field that is both **static** and **final** has only one piece of storage that cannot be changed.
- The keyword final can also be applied to **methods**, and **classes**.
- A final variable that is **not initialized** at the time of declaration is known as **blank final variable**. We can initialize using constructors only.
- A **static final variable** that is not initialized at the time of declaration is known as static blank final variable. It can be initialized only in static block.

```
final int FILE_NEW = 1;
final int FILE_OPEN = 2;
final int FILE_SAVE = 3;
final int FILE_SAVEAS = 4;
final int FILE_QUIT = 5;
```

```
class A{
    static final int data;//static blank final variable
    static{ data=50;}
    public static void main(String args[]){
        System.out.println(A.data);
    }
}
```

final variable

- If you declare any **parameter** as final, you cannot change the value of it.

```
class Bike11{  
    int cube(final int n){  
        n=n+2; //can't be changed as n is final  
        n*n*n;  
    }  
    public static void main(String args[]){ Bike11 b=new  
        Bike11();  
        b.cube(5);  
    }  
}
```

Can we declare a constructor final?

No, because constructor is never inherited.

```
class finalvar{  
    final int x=90;           //final variable  
    void run(){  
        x=400;  
    }  
    public static void main(String args[])  
    {  
        finalvar obj=new finalvar();  
        obj.run();  
    }  
} //end of class
```

Output: Compile Time Error

- If you make any method as final, you **cannot** override it.
- Is final method inherited?

Yes, final method is inherited but you cannot override it.

```
class Bike{
    final void run(){
        System.out.println("running");
    }
}
class Honda extends Bike{
    void run(){
        System.out.println("running safely with 100kmph");
    }
    public static void main(String args[]){
        Honda honda= new Honda();
        honda.run();
    }
}
```

Output:Compile Time Error

final class

- If you make any class as final, you cannot extend it.

```
class Bike{
    final void run(){
        System.out.println("running");
    }
}
class Honda extends Bike{
    void run(){
        System.out.println("running safely with 100kmph");
    }
}
class finalMethod
{
    public static void main(String args[]){
        Honda h= new Honda();
        h.run();
    }
}
```

String Class

- The first thing, strings are actually an object of type String.
- Even string **constants** are actually String objects. Example

```
System.out.println("This is a String, too");
```
- objects of type String are **immutable**; once a String object is created, its contents cannot be altered.
 - If you need to change a string, you can always create a new one that contains the modifications.
 - Java defines a peer class of String, called **StringBuffer**, which allows strings to be altered.
- + is used to concatenate two strings.
- Test two strings for equality by using **equals()**.
- length of a string by calling the **length()** method.
- character at a specified index within a string by calling **charAt()**.

// Demonstrating some String methods.

```
class StringDemo2 {  
    public static void main(String args[]) {  
        String strOb1 = "First String";  
        String strOb2 = "Second String";  
        String strOb3 = strOb1;  
        System.out.println("Length of strOb1: " + strOb1.length());  
        System.out.println("Char at index 3 in strOb1: " + strOb1.charAt(3));  
        if(strOb1.equals(strOb2))  
            System.out.println("strOb1 == strOb2");  
        else  
            System.out.println("strOb1 != strOb2");  
        if(strOb1.equals(strOb3))  
            System.out.println("strOb1 == strOb3");  
        else  
            System.out.println("strOb1 != strOb3");  
    }  
}
```

OUTPUT

```
Length of strOb1: 12  
Char at index 3 in strOb1: s  
strOb1 != strOb2  
strOb1 == strOb3
```

command line

arguments

- Sometimes you will want to pass information into a program when you run it. This is accomplished by passing *command-line arguments to main()*.
- *A command-line argument is the information that directly follows the program's name on the command line when it is executed.*
- To access the command-line arguments inside a Java program, they are stored as strings in a **String array** passed to the **args** parameter of main().
- The first command-line argument is stored at **args[0]**, the second at **args[1]**, and so on.
- *All command-line arguments are passed as **strings**.*

command line arguments

```
public class CommandLine
{
    public static void main(String args[])
    {
        for(int i = 0; i<args.length; i++)
        {
            System.out.println("args[" + i + "]: " + args[i]);
        }
    }
}
```

d:/mahesh>javac CommandLine.java

d:/mahesh>java CommandLine this is a command line 200 -100

Output

```
args[0]: this
args[1]: is
args[2]: a
args[3]: command
args[4]: line
args[5]: 200
args[6]: -100
```

Inheritance

- *Java inheritance* refers to the ability in Java for one class to **inherit** from another class.
- In Java this is also called **extending** a class. One class can *extend* another class and thereby *inherit* from that class.
- When one class inherits from another class in Java, the two classes take on certain roles. The class that extends (inherits from another class) is the **subclass** and the class that is being extended (the class being inherited from) is the **superclass**.
- In other words, the subclass extends the superclass. Or, the subclass inherits from the superclass.
- Another commonly used term for inheritance is **specialization** and **generalization**. A subclass is a specialization of a superclass, and a superclass is a generalization of one or more subclasses.
- **Inheritance is a Method of Code Reuse.**
- **extends** is the keyword used to inherit the properties of a class.

synt
ax

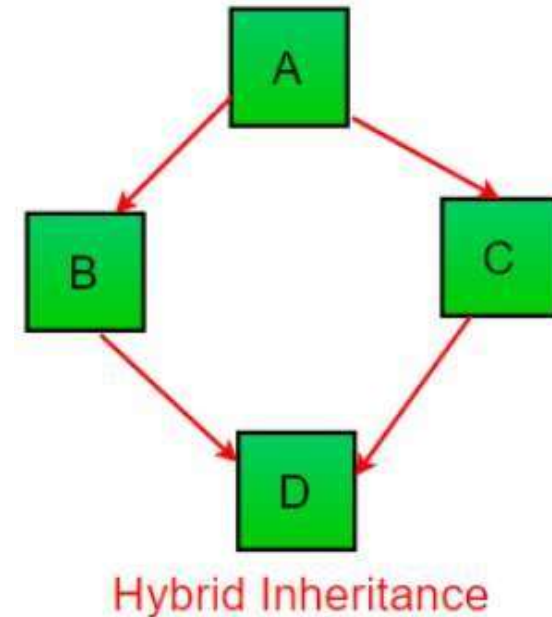
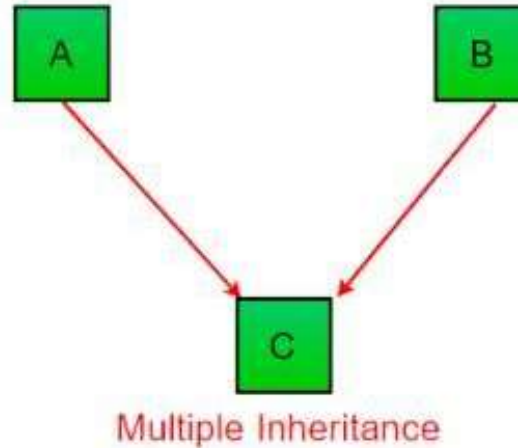
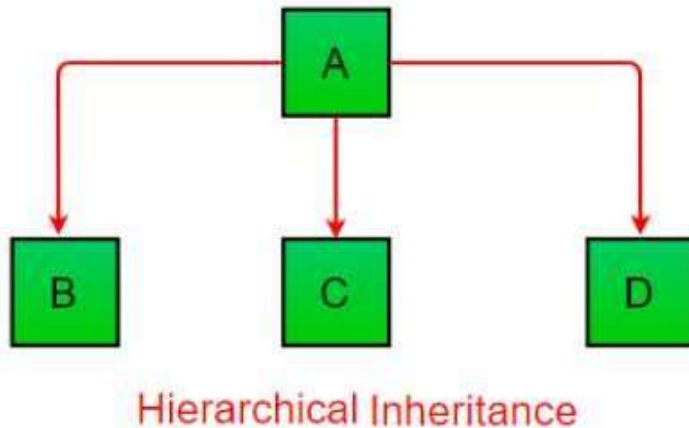
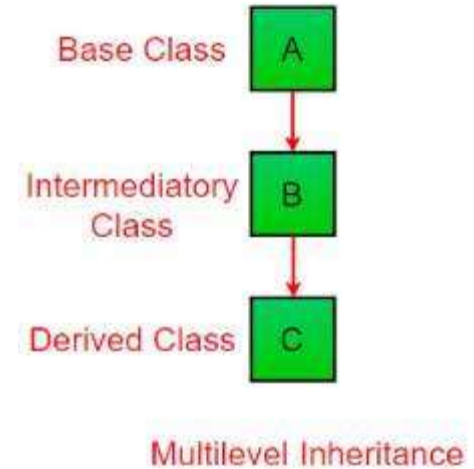
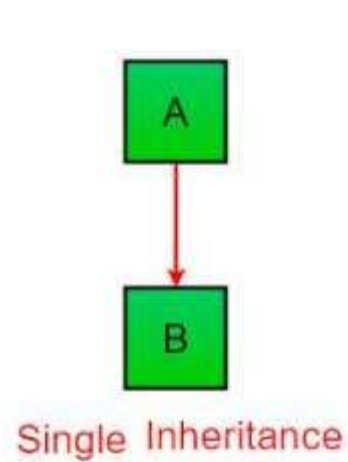
```
class Subclass-name extends Superclass-name  
{  
    //methods and fields  
}
```

What is Inherited?

- When a subclass extends a superclass in Java, all **protected** and **public** fields and methods of the superclass are inherited by the subclass.
- By *inherited* is meant that these fields and methods are **part of** the subclass, as if the subclass had declared them itself.
- protected and public fields can be called and referenced just like the methods declared directly in the subclass.
- Fields and methods with **default** (package) access modifiers can be accessed by subclasses only if the subclass is located in the same package as the superclass.
- Private fields and methods of the superclass can **never be** referenced directly by subclasses. They can, however, be referenced indirectly via methods reachable from the subclass (e.g default (package), protected and public methods).
- **Constructors are not inherited by subclasses**, but a subclass constructor must call a constructor in the superclass.

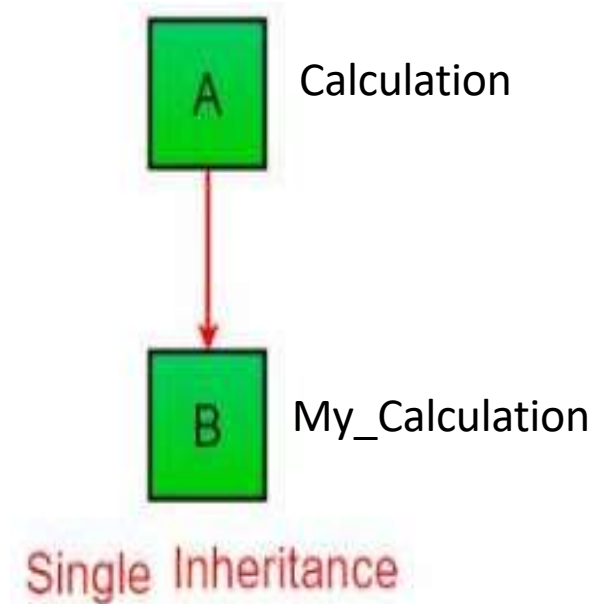
Types of Inheritance in Java

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (Through Interfaces)
5. Hybrid Inheritance(Through Interfaces)



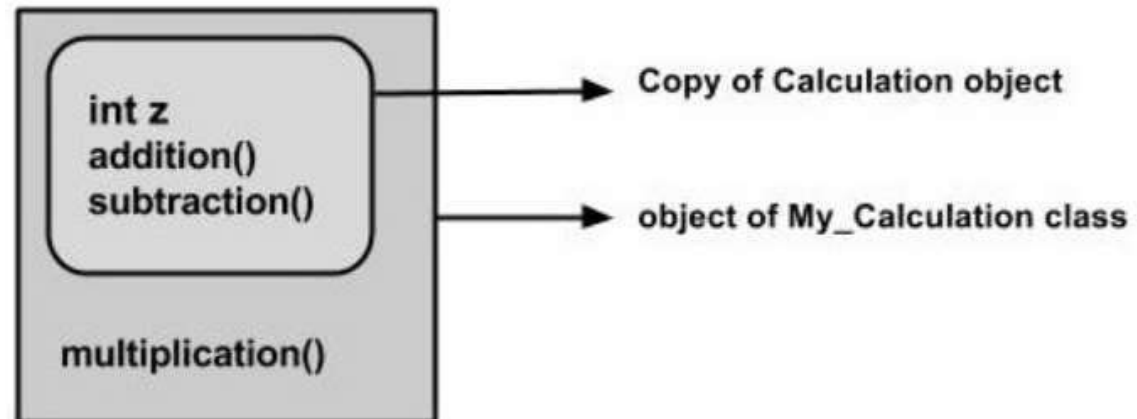
Single Inheritance

```
class Calculation {  
    int z;  
  
    public void addition(int x, int y) {  
        z = x + y;  
        System.out.println("The sum of the given numbers:"+z);  
    }  
  
    public void Subtraction(int x, int y) {  
        z = x - y;  
        System.out.println("The difference between the given numbers:"+z);  
    }  
}
```




```
public class My_Calculation extends Calculation {  
    public void multiplication(int x, int y) {  
        z = x * y;  
        System.out.println("The product of the given numbers:"+z);  
    }  
}
```

```
public static void main(String args[]) {  
    int a = 20, b = 10;  
    My_Calculation demo = new My_Calculation();  
    demo.addition(a, b);  
    demo.Subtraction(a, b);  
    demo.multiplication(a, b);  
}  
}
```



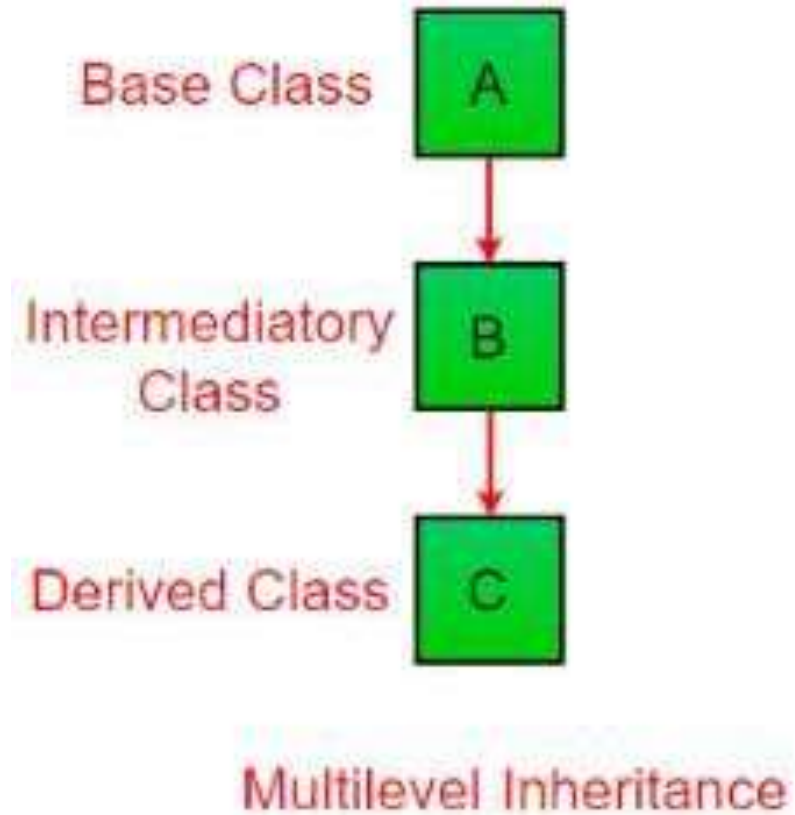
```
// A simple example of inheritance.
// Create a superclass.
class A {
    int i, j;
    void showij() {
        System.out.println("i and j: " + i + " " + j);
    }
}

// Create a subclass by extending class A.
class B extends A {
    int k;
    void showk() {
        System.out.println("k: " + k);
    }
    void sum() {
        System.out.println("i+j+k: " + (i+j+k));
    }
}
```

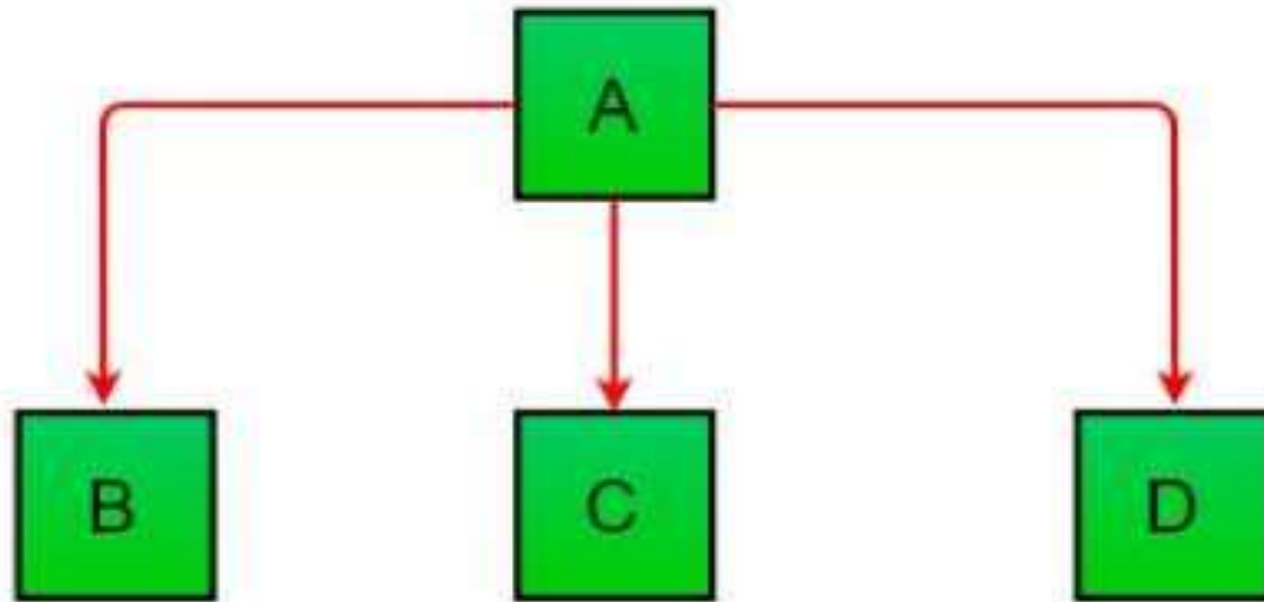
```
class SimpleInheritance {
    public static void main(String args[]) {
        A superOb = new A();
        B subOb = new B();
        // The superclass may be used by itself.
        superOb.i = 10;
        superOb.j = 20;
        System.out.println("Contents of superOb: ");
        superOb.showij();
        System.out.println();
        /* The subclass has access to all public members
        of its superclass. */
        subOb.i = 7;
        subOb.j = 8;
        subOb.k = 9;
        System.out.println("Contents of subOb: ");
        subOb.showij();
        subOb.showk();
        System.out.println();
        System.out.println("Sum of i, j and k in subOb:");
        subOb.sum();
    }
}
```

Multilevel Inheritance

- Lab program



Hierarchical Inheritance



Hierarchical Inheritance

//Base class

Example - 1

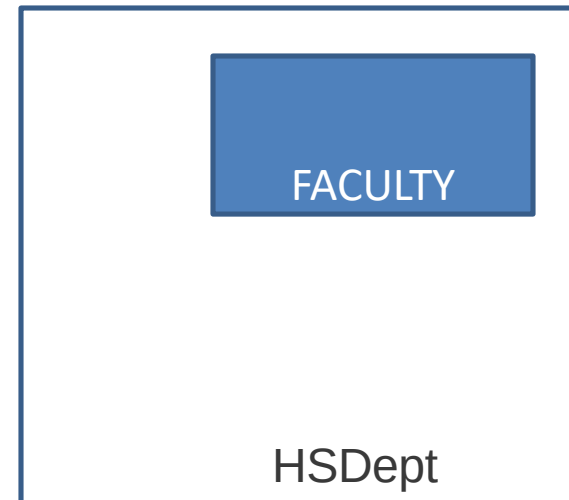
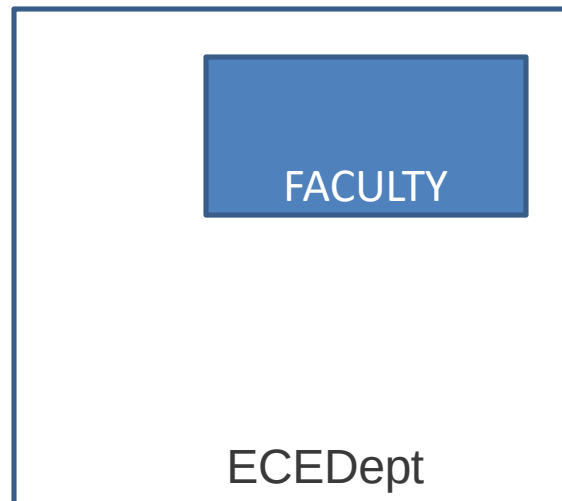
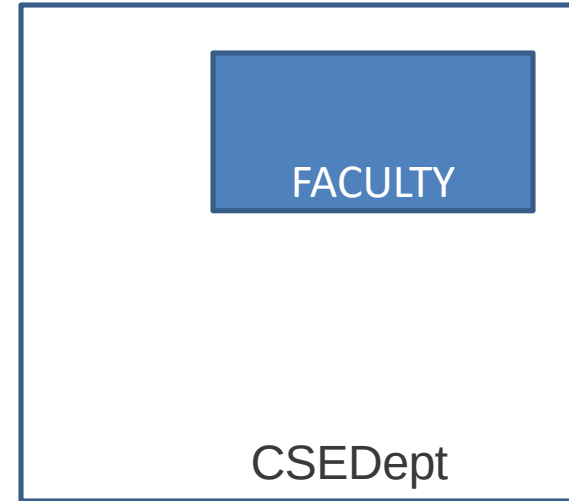
```
class Faculty
{
    public void JP() {
        System.out.println("Java programming...");
    }
    void CP() {
        System.out.println("C Programming...");
    }
    void CPP() {
        System.out.println("C ++ Programming...");
    }
}
```

```
// Inherits feature of Faculty class
class CSEDept extends Faculty{
    public void learn() {
        System.out.println("Computer Science Department ...");
    }
}

// Inherits feature of same Faculty class
class ECEDept extends Faculty{
    public void learn() {
        System.out.println("\nECE Department...");
    }
}

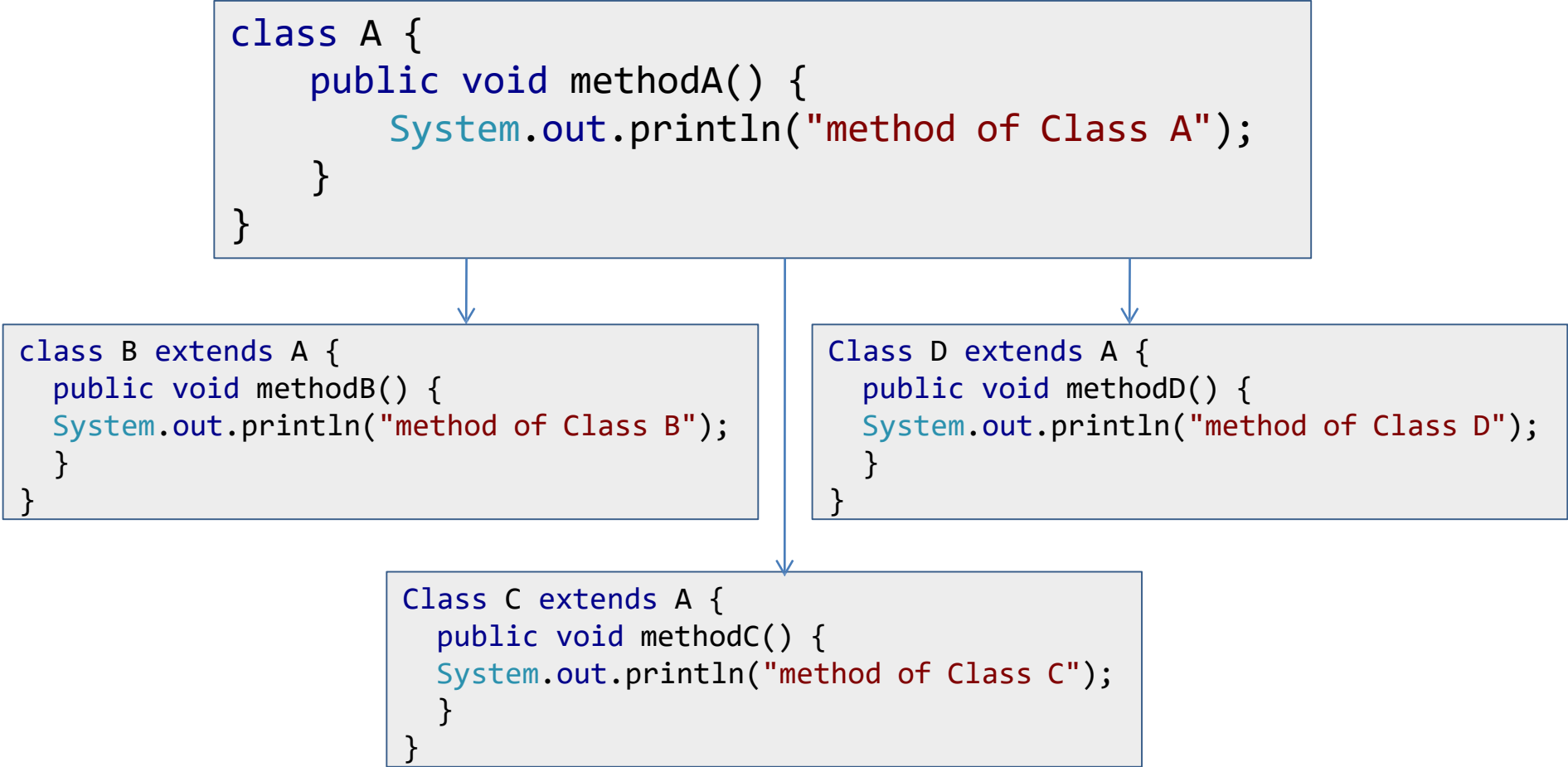
// Inherits feature of same Faculty class
class HSDept extends Faculty{
    public void learn() {
        System.out.println("\nHS Department...");
    }
}
```

```
public class HierarchicalInheritance {  
    public static void main(String[] args) {  
        CSEDept c = new CSEDept();  
        c.learn();  
        c.JP();  
  
        ECEDept e = new ECEDept();  
        e.learn();  
        e.CP();  
  
        HSDept h = new HSDept();  
        h.learn();  
        h.CPP();  
    }  
}
```



Example - 2

```
class A {  
    public void methodA() {  
        System.out.println("method of Class A");  
    }  
}
```



```
class B extends A {  
    public void methodB() {  
        System.out.println("method of Class B");  
    }  
}
```

```
Class D extends A {  
    public void methodD() {  
        System.out.println("method of Class D");  
    }  
}
```

```
Class C extends A {  
    public void methodC() {  
        System.out.println("method of Class C");  
    }  
}
```



```
class JavaExample {  
public static void main(String args[]) {  
    B obj1 = new B();  
    C obj2 = new C();  
    D obj3 = new D(); //All classes can access the method of class A  
  
    obj1.methodA();  
    obj1.methodB();  
  
    obj2.methodA();  
    obj2.methodC();  
  
    obj3.methodA(); }  
    obj3.methodD();  
}
```

private -

Inheritance

- Although a subclass includes all of the members of its superclass, it **cannot access** those members of the superclass that have been declared as **private**.
- *A class member that has been declared as private will remain private to its class. It is not accessible by any code outside its class, including subclasses.*

```
class A {  
    int i; // public by default  
    private int j; // private to A  
    void setij(int x, int y) {  
        i = x;  
        j = y;  
    }  
}  
  
// A's j is not accessible here.  
class B extends A {  
    int total;  
    void sum() {  
        total = i + j; // ERROR, j is not accessible here  
    }  
}
```

A Superclass Variable
Can Reference a
Subclass Object

- A reference variable of a superclass can be assigned a reference to any subclass derived from that superclass.

```
class Data {  
    int data1;  
    int data2;  
}  
class NewData extends Data{  
    int data3;  
    int data4;  
}  
public class Javaapp {  
    public static void main(String[] args) {  
        Data obj = new NewData();  
        obj.data1 = 50;  
        obj.data2 = 100;  
        System.out.println("obj.data1 = "+obj.data1);  
        System.out.println("obj.data2 = "+obj.data2);  
    }  
}
```

Using super

- Whenever a subclass needs to refer to its immediate superclass, it can do so by use of the keyword super.
- super has two general forms.
 - The **first** calls the superclass' constructor.
 - The **second** is used to access a member of the superclass that has been hidden by a member of a subclass.

1. Using super to Call Superclass Constructors

- A subclass can call a constructor defined by its superclass by use of the following form of **super**:
super(arg-list);
- Here, *arg-list* specifies any arguments needed by the constructor in the superclass.
- **super()** must always be the first statement executed inside a subclass' constructor.
- When a subclass calls **super()**, it is calling the constructor of its immediate superclass.

```
// A complete implementation of BoxWeight.
class Box {
private double width;
private double height;
private double depth;
// construct clone of an object
Box(Box ob) { // pass object to constructor
width = ob.width;
height = ob.height;
depth = ob.depth;
}
// constructor used when all dimensions specified
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
// constructor used when no dimensions specified
Box() {
width = -1; // use -1 to indicate
height = -1; // an uninitialized
depth = -1; // box
}
```

```
// constructor used when cube is created
Box(double len) {
width = height = depth = len;
}
// compute and return volume
double volume() {
return width * height * depth;
}
}
```

// BoxWeight now fully implements all constructors.

```
class BoxWeight extends Box {  
    double weight;           // weight of box  
    // construct clone of an object  
    BoxWeight(BoxWeight ob) { // pass object to constructor  
        super(ob);  
        weight = ob.weight;  
    }  
    // constructor when all parameters are specified  
    BoxWeight(double w, double h, double d, double m) {  
        super(w, h, d);      // call superclass constructor  
        weight = m;  
    }  
    // default constructor  
    BoxWeight() {  
        super();  
        weight = -1;  
    }  
    // constructor used when cube is created  
    BoxWeight(double len, double m) {  
        super(len);  
        weight = m;  
    }  
}
```



```

class DemoSuper {
    public static void main(String args[]) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
        BoxWeight mybox3 = new BoxWeight();
        BoxWeight mycube = new BoxWeight(3, 2);
        BoxWeight myclone = new BoxWeight(mybox1);
        double vol;
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        System.out.println("Weight of mybox1 is " + mybox1.weight);
        System.out.println();
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        System.out.println("Weight of mybox2 is " + mybox2.weight);
        System.out.println();
        vol = mybox3.volume();
        System.out.println("Volume of mybox3 is " + vol);
        System.out.println("Weight of mybox3 is " + mybox3.weight);
        System.out.println();
        vol = myclone.volume();
        System.out.println("Volume of myclone is " + vol);
        System.out.println("Weight of myclone is " + myclone.weight);
        System.out.println();
        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
        System.out.println("Weight of mycube is " + mycube.weight);
        System.out.println();
    }
}

```

OUTPUT

```

Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3
Volume of mybox2 is 24.0
Weight of mybox2 is 0.076
Volume of mybox3 is -1.0
Weight of mybox3 is -1.0
Volume of myclone is 3000.0
Weight of myclone is 34.3
Volume of mycube is 27.0
Weight of mycube is 2.0

```

2. A Second Use for super

- The second form of super acts somewhat like **this**, except that it always refers to the superclass of the subclass in which it is used.

super.member

- Here, *member* can be either a method or an instance variable.
- This second form of super is most applicable to situations in which member names of a **subclass** **hide members** by the same name in the superclass.

// Using super to overcome name hiding.

```
class A {  
    int i;  
}  
class B extends A {  
    int i;          // this i hides the i in A  
    B(int a, int b) {  
        super.i = a;    // i in A  
        i = b;          // i in B  
    }  
    void show() {  
        System.out.println("i in superclass: " + super.i);  
        System.out.println("i in subclass: " + i);  
    }  
}  
class UseSuper {  
    public static void main(String args[]) {  
        B subOb = new B(1, 2);  
        subOb.show();  
    }  
}
```

OUTPUT

i in superclass: 1

i in subclass: 2

When Constructors

Are Called

- When a class hierarchy is created, in what order are the constructors for the classes that make up the hierarchy called?
- The answer is that in a class hierarchy, constructors are called in **order of derivation**, from **superclass to subclass**.
- Further, since `super( )` must be the first statement executed in a subclass' constructor, this order is the same whether or not `super( )` is used.
- If `super( )` is **not used**, then the default or parameterless constructor of each superclass will be executed.

```
// Create a super class.
class A {
A() {
    System.out.println("Inside A's constructor.");
}
}

// Create a subclass by extending class A.
class B extends A {
B() {
    System.out.println("Inside B's constructor.");
}
}

// Create another subclass by extending B.
class C extends B {
C() {
    System.out.println("Inside C's constructor.");
}
}

class CallingCons {
public static void main(String args[]) {
C c = new C();
}
}
```

OUTPUT

```
Inside A's constructor
Inside B's constructor
Inside C's constructor
```

Method Overriding

- In a class hierarchy, when a method in a subclass has the **same name and type signature** as a method in its superclass, then the method in the subclass is said to *override the method in* the superclass.
- When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass.
- The version of the method defined by the superclass will be **hidden**.

```
// Method overriding.
class A {
    int i, j;
    A(int a, int b) {
        i = a;
        j = b;
    }
    // display i and j
    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}

class B extends A {
    int k;
    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }
    // display k – this overrides show() in A
    void show() {
        System.out.println("k: " + k);
    }
}
```

```
class Override {
    public static void main(String args[]) {
        B subOb = new B(1, 2, 3);
        subOb.show(); // this calls show() in B
    }
}
```

OUTPUT
k: 3

If you wish to access the superclass version of an overridden method,

```
class B extends A {  
    int k;  
    B(int a, int b, int c) {  
        super(a, b);  
        k = c;  
    }  
    void show() {  
        super.show(); // this calls A's show()  
        System.out.println("k: " + k);  
    }  
}
```


Polymorphism in Java

- **Polymorphism in Java** is a concept by which we can perform a *single action in different ways*.
- Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.
- There are **two** types of polymorphism in Java:
 - compile-time polymorphism (method overloading) and
 - runtime polymorphism (method overriding).

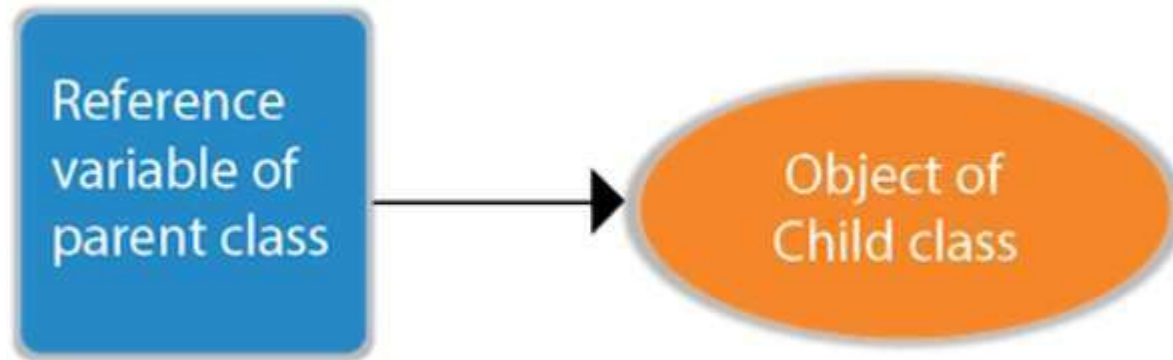
Runtime polymorphism

- Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.
- Dynamic method dispatch is important because this is how Java implements run-time polymorph
- In this process, an **overridden** method is called through the **reference variable of a superclass**.
- The determination of the method to be called is based on the object being referred to by the reference variable.
- If the reference variable of Parent class refers to the object of Child class, it is known as **upcasting**.
- At run-time, it depends on the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.

- In Java, we can override methods only, **not the variables(data members)**, so runtime polymorphism **cannot** be achieved by data members.
- If the reference variable of Parent class refers to the object of Child class, it is known as **upcasting**.

```
class A{  
}  
class B extends A{  
}
```

```
A a=new B();    //upcasting
```



// A Java program to illustrate Dynamic Method Dispatch using hierarchical inheritance

class A

{

void m1()

{

System.out.println("Inside A's m1 method");

}

}

class B extends A

{

// overriding m1()

void m1()

{

System.out.println("Inside B's m1 method");

}

}

class C extends A

{

// overriding m1()

void m1()

{

System.out.println("Inside C's m1 method");

}

}

```
// Driver class
class Dispatch
{
    public static void main(String args[])
    {
        // object of type A
        A a = new A();
        // object of type B
        B b = new B();
        // object of type C
        C c = new C();
        // obtain a reference of type A
        A ref;

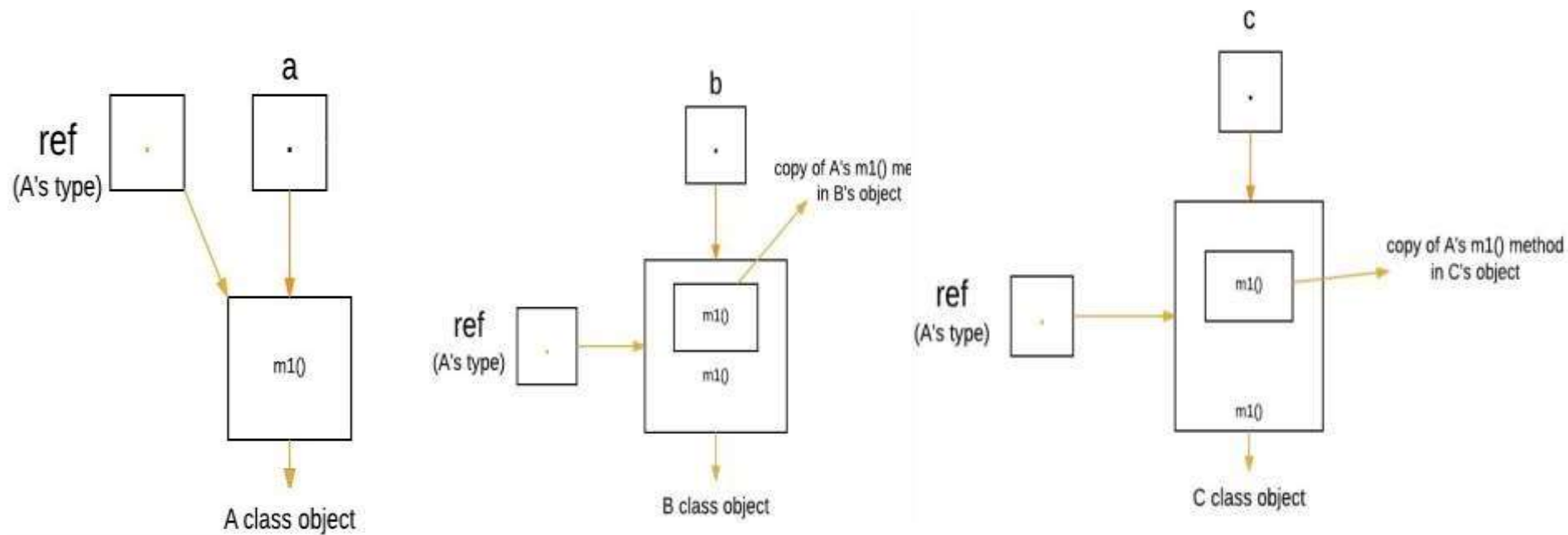
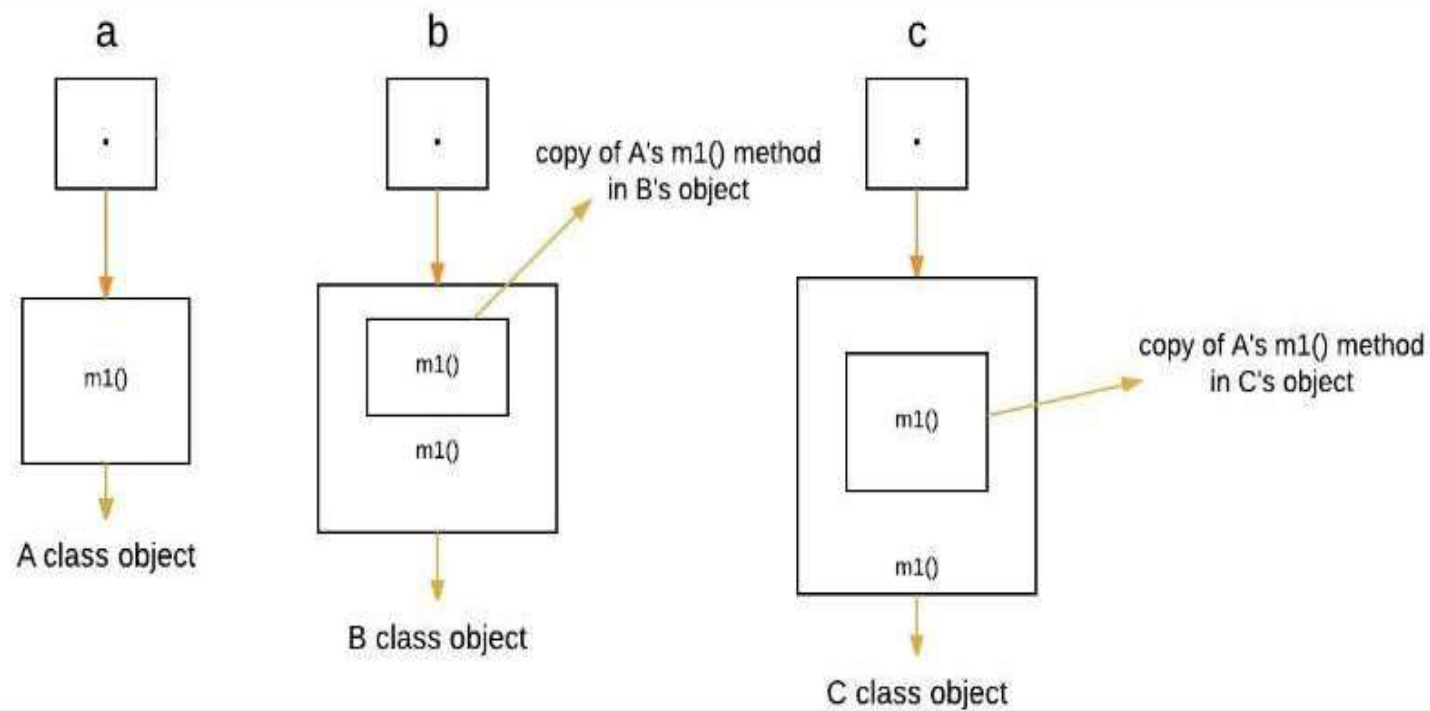
        ref = a;           // ref refers to an A object
        ref.m1();           // calling A's version of m1()

        ref = b;           // now ref refers to a B object
        ref.m1();           // calling B's version of m1()

        ref = c;           // now ref refers to a C object
        ref.m1();           // calling C's version of m1()
    }
}
```

Output:

```
Inside A's m1 method
Inside B's m1 method
Inside C's m1 method
```



Static vs Dynamic binding

- Static binding is done during compile-time while dynamic binding is done during run-time.
- private, final and static methods and variables uses static binding and bonded by compiler while overridden methods are bonded during runtime based upon type of runtime object

Why Overridden

Methods?

- Overridden methods allow Java to support run-time polymorphism.
- Polymorphism is essential to object-oriented programming for one reason: it allows a general class to specify methods that will be common to all of its derivatives, while allowing subclasses to define the specific implementation of some or all of those methods.
- Overridden methods are another way that Java implements the “one interface, multiple methods” aspect of polymorphism.
- Dynamic, run-time polymorphism is one of the most powerful mechanisms that objectoriented design brings to bear on code reuse and robustness.
- *C++ or C# will recognize that overridden methods in Java are similar to **virtual functions** in those languages.*

Abstract Classes

- A class which is declared with the abstract keyword is known as an abstract class in Java. It can have abstract and non-abstract methods (method with the body).
- **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.
- Another way, it shows only essential things to the user and hides the internal details, for **example**, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.
- Abstraction lets you focus on what the object does instead of how it does it.
- There are two ways to achieve abstraction in java
 - Abstract class (0 to 100%)
 - Interface (100%)

```
abstract class class-name  
{  
  
    non-abstract/regular methods    //defined within { }  
    {  
    }  
  
    abstract methods();    //ending with a semicolon;  
    }
```

An abstract class may contain a mix of non-abstract methods and abstract methods, or only abstract methods ,or only non-abstract regular methods.

Rules for Java Abstract class



1

An abstract class must be declared with an abstract keyword.

2

It can have abstract and non-abstract methods.

3

It cannot be instantiated.

4

It can have final methods

5

It can have constructors and static methods also.

- An abstract class must be declared with an **abstract** keyword.
- It can have abstract and non-abstract methods.
- It **cannot** be instantiated. An abstract class cannot be directly instantiated with the new operator. Such objects would be useless, because an abstract class is not fully defined.
- It can have constructors and static methods also.
- Cannot declare abstract constructors, or abstract static methods.
- It can have final methods which will force the subclass not to change the body of the method.
- If there is an abstract method in a class, that class **must be abstract**.
- If you are extending an abstract class that has an abstract method, you must either provide the implementation of the method or make this class abstract.

Abstract Method in Java

- A method which is declared as abstract and **does not have implementation** is known as an abstract method.

abstract type name(parameter-list);

abstract void printStatus();

```
abstract class Bank{  
    abstract int  
    getRateOfInterest();  
}
```

```
class SBI extends Bank{  
    int getRateOfInterest(){  
        return 7;}  
}
```

```
class PNB extends Bank{  
    int getRateOfInterest(){  
        return 8;}  
}
```

```
class TestBank{  
    public static void main(String args[]){  
        Bank b;  
        b=new SBI();  
        System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");  
  
        b=new PNB();  
        System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");  
  
    }  
}
```

OUTPUT

Rate of Interest is: 7 %

Rate of Interest is: 8 %

//Example of an abstract class that has abstract and non-abstract methods

```
abstract class Bike{  
    Bike(){  
        System.out.println("bike is created");}  
    abstract void run();  
    void changeGear(){  
        System.out.println("gear changed");}  
}
```

//Creating a Child class which inherits Abstract class

```
class Honda extends Bike{  
    void run(){  
        System.out.println("running safely..");}  
}
```

//Creating a Test class which calls abstract and non-abstract methods

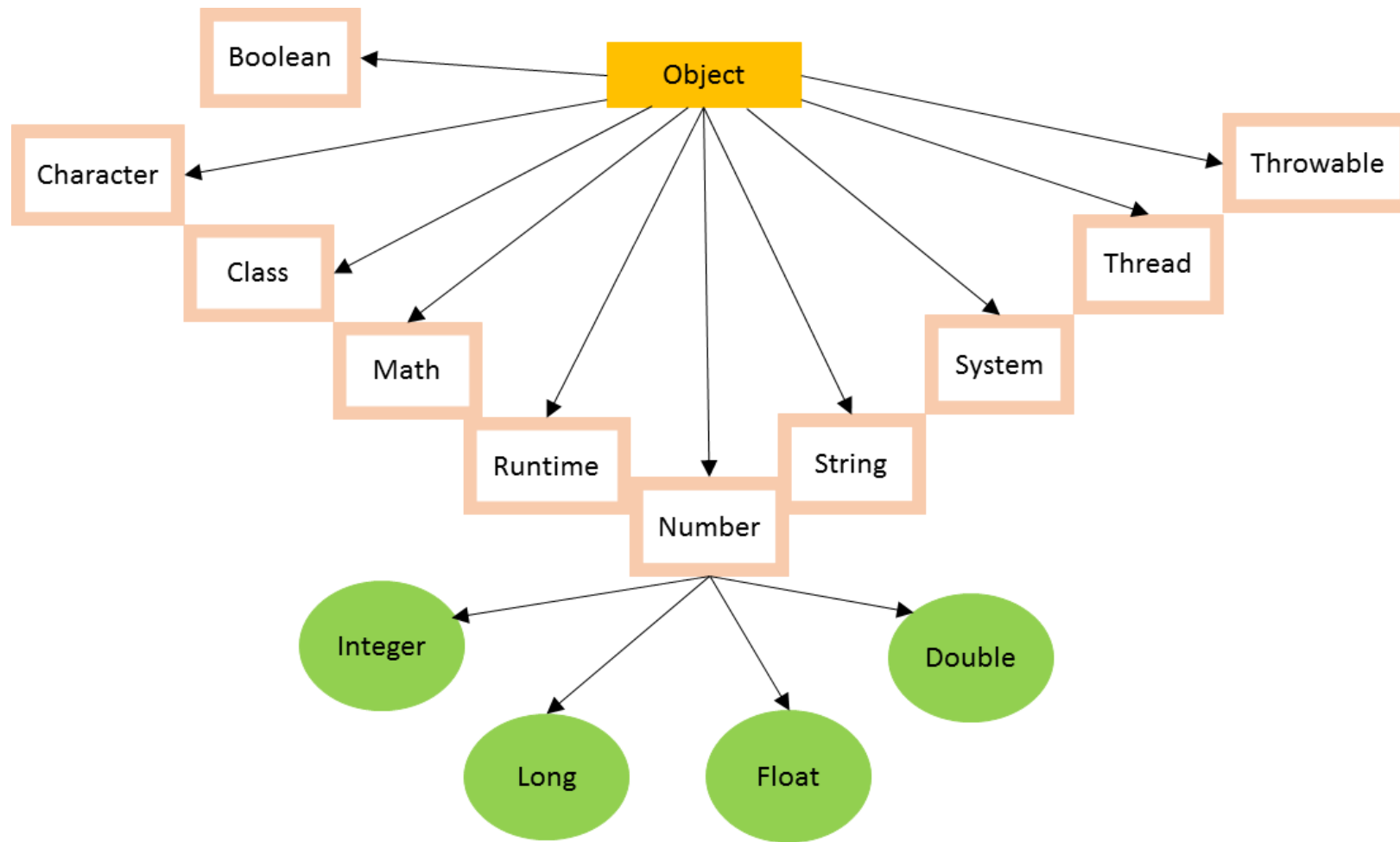
```
class TestAbstraction2{  
    public static void main(String args[]){  
        Bike obj = new Honda();  
        obj.run();  
        obj.changeGear();  
    }  
}
```

OUTPUT

bike is created
running safely..
gear changed

The Object Class

- There is one special class, **Object**, defined by Java.
- All other classes are **subclasses** of Object. That is, Object is a superclass of all other classes. This means that a reference variable of type Object can refer to an object of any other class.
- Also, since arrays are implemented as classes, a variable of type Object can also refer to any array.



Method	Purpose
Object clone()	Creates a new object that is the same as the object being cloned.
boolean equals(Object <i>object</i>)	Determines whether one object is equal to another.
void finalize()	Called before an unused object is recycled.
Class getClass()	Obtains the class of an object at run time.
int hashCode()	Returns the hash code associated with the invoking object.
void notify()	Resumes execution of a thread waiting on the invoking object.
void notifyAll()	Resumes execution of all threads waiting on the invoking object.
String toString()	Returns a string that describes the object.
void wait() void wait(long <i>milliseconds</i>) void wait(long <i>milliseconds</i> , int <i>nanoseconds</i>)	Waits on another thread of execution.

Inheritance

- *Java inheritance* refers to the ability in Java for one class to **inherit** from another class.
- In Java this is also called **extending** a class. One class can *extend* another class and thereby *inherit* from that class.
- When one class inherits from another class in Java, the two classes take on certain roles. The class that extends (inherits from another class) is the **subclass** and the class that is being extended (the class being inherited from) is the **superclass**.
- In other words, the subclass extends the superclass. Or, the subclass inherits from the superclass.
- Another commonly used term for inheritance is **specialization** and **generalization**. A subclass is a specialization of a superclass, and a superclass is a generalization of one or more subclasses.
- **Inheritance is a Method of Code Reuse.**
- **extends** is the keyword used to inherit the properties of a class.

synt

ax

```
class Subclass-name extends Superclass-name
```

```
{
```

```
    //methods and fields
```

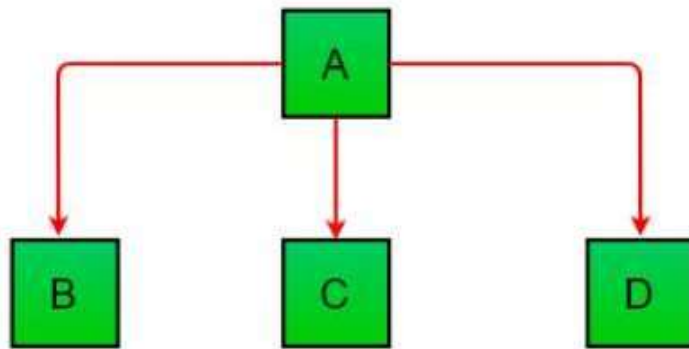
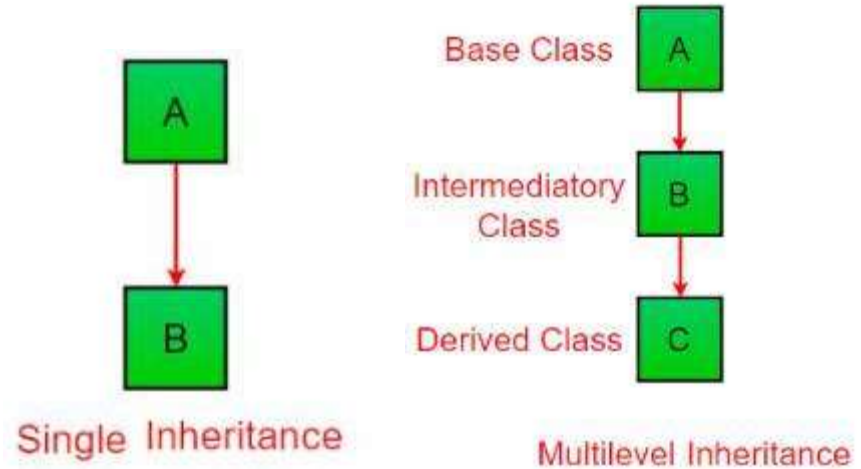
```
}
```

What is Inherited?

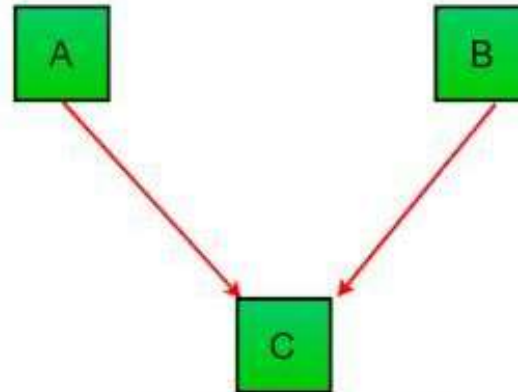
- When a subclass extends a superclass in Java, all **protected** and **public** fields and methods of the superclass are inherited by the subclass.
- By *inherited* is meant that these fields and methods are **part of** the subclass, as if the subclass had declared them itself.
- protected and public fields can be called and referenced just like the methods declared directly in the subclass.
- Fields and methods with **default** (package) access modifiers can be accessed by subclasses only if the subclass is located in the same package as the superclass.
- Private fields and methods of the superclass can **never be** referenced directly by subclasses. They can, however, be referenced indirectly via methods reachable from the subclass (e.g default (package), protected and public methods).
- **Constructors are not inherited by subclasses**, but a subclass constructor must call a constructor in the superclass.

Types of Inheritance in Java

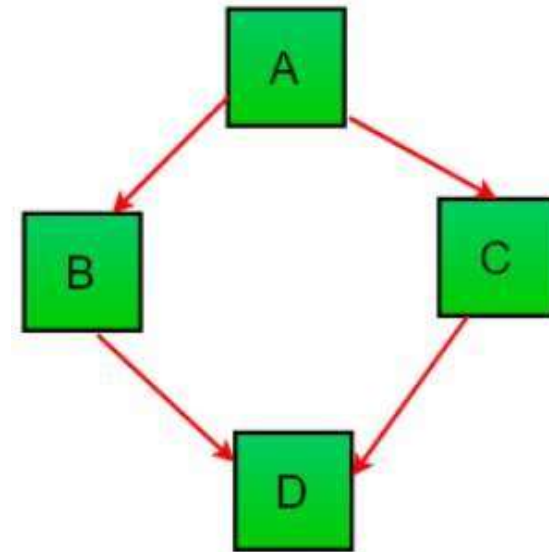
1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (Through **Interfaces**)
5. Hybrid Inheritance(Through **Interfaces**)



Hierarchical Inheritance



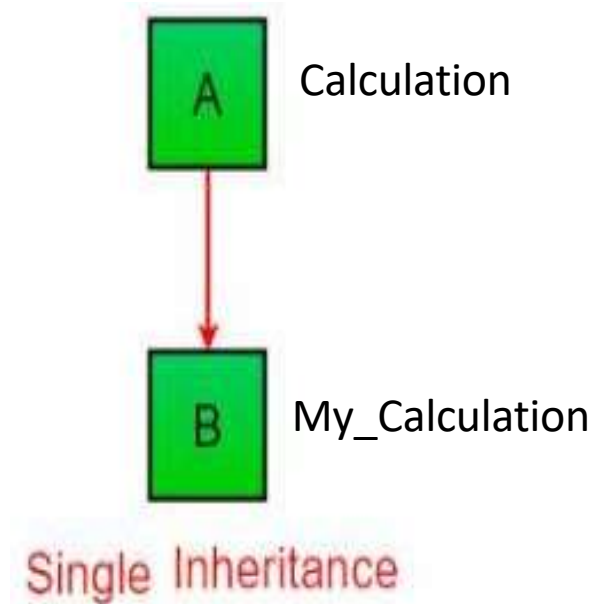
Multiple Inheritance



Hybrid Inheritance

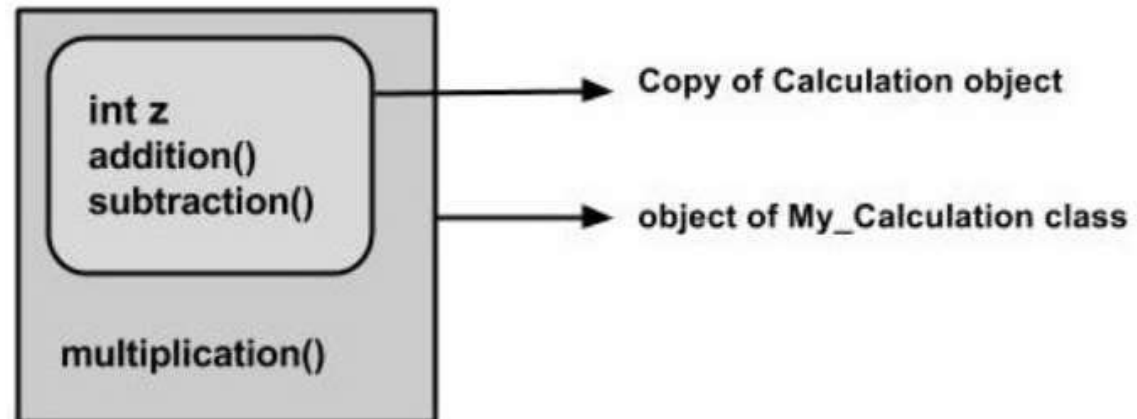
Single Inheritance

```
class Calculation {  
    int z;  
  
    public void addition(int x, int y) {  
        z = x + y;  
        System.out.println("The sum of the given numbers:"+z);  
    }  
  
    public void Subtraction(int x, int y) {  
        z = x - y;  
        System.out.println("The difference between the given numbers:"+z);  
    }  
}
```



```
public class My_Calculation extends Calculation {  
    public void multiplication(int x, int y) {  
        z = x * y;  
        System.out.println("The product of the given numbers:"+z);  
    }  
}
```

```
public static void main(String args[]) {  
    int a = 20, b = 10;  
    My_Calculation demo = new My_Calculation();  
    demo.addition(a, b);  
    demo.Subtraction(a, b);  
    demo.multiplication(a, b);  
}  
}
```



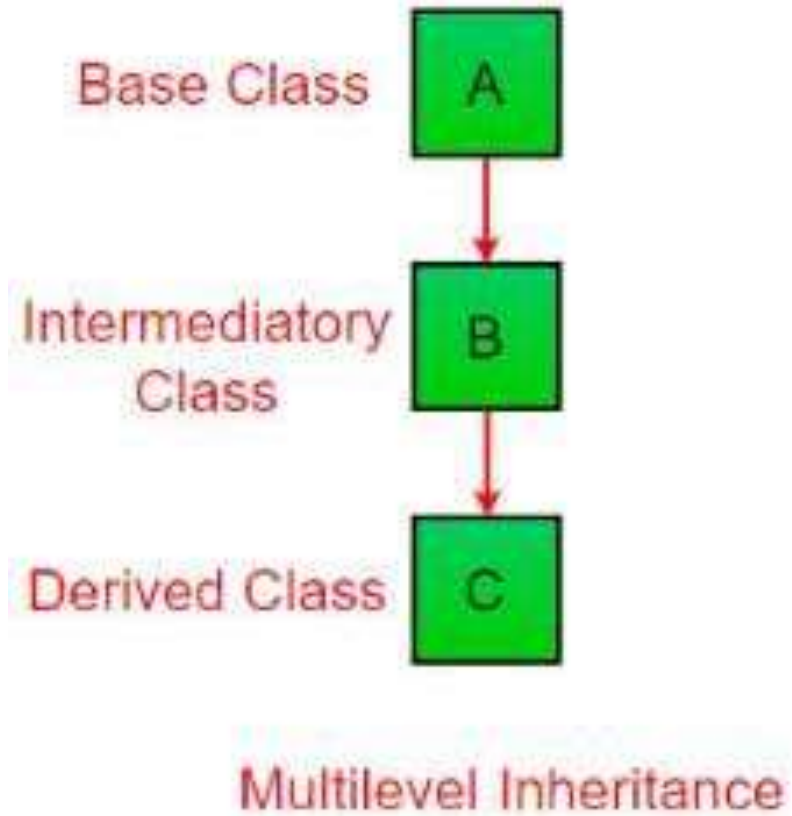
```
// A simple example of inheritance.
// Create a superclass.
class A {
    int i, j;
    void showij() {
        System.out.println("i and j: " + i + " " + j);
    }
}

// Create a subclass by extending class A.
class B extends A {
    int k;
    void showk() {
        System.out.println("k: " + k);
    }
    void sum() {
        System.out.println("i+j+k: " + (i+j+k));
    }
}
```

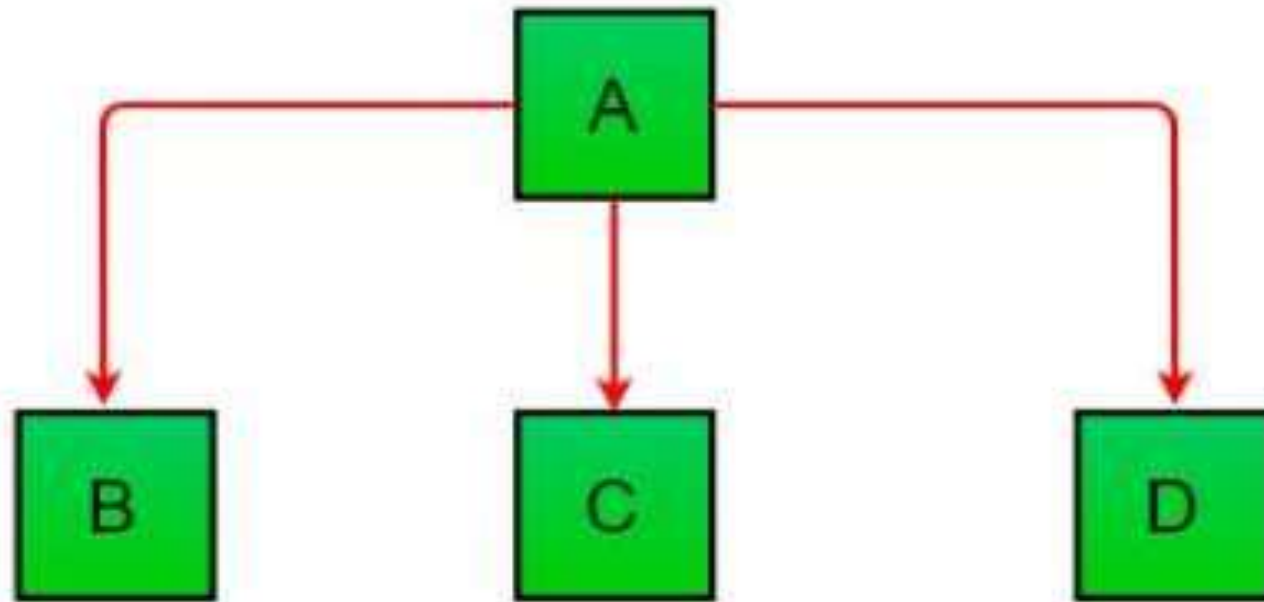
```
class SimpleInheritance {
    public static void main(String args[]) {
        A superOb = new A();
        B subOb = new B();
        // The superclass may be used by itself.
        superOb.i = 10;
        superOb.j = 20;
        System.out.println("Contents of superOb: ");
        superOb.showij();
        System.out.println();
        /* The subclass has access to all public members
        of its superclass. */
        subOb.i = 7;
        subOb.j = 8;
        subOb.k = 9;
        System.out.println("Contents of subOb: ");
        subOb.showij();
        subOb.showk();
        System.out.println();
        System.out.println("Sum of i, j and k in subOb:");
        subOb.sum();
    }
}
```


Multilevel Inheritance

- Lab program



Hierarchical Inheritance



Hierarchical Inheritance

//Base class

Example - 1

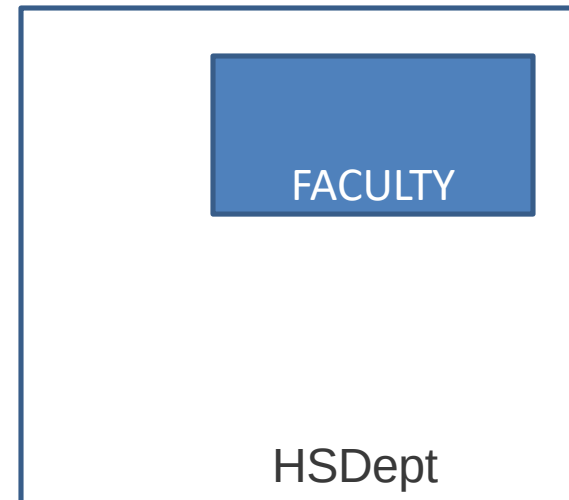
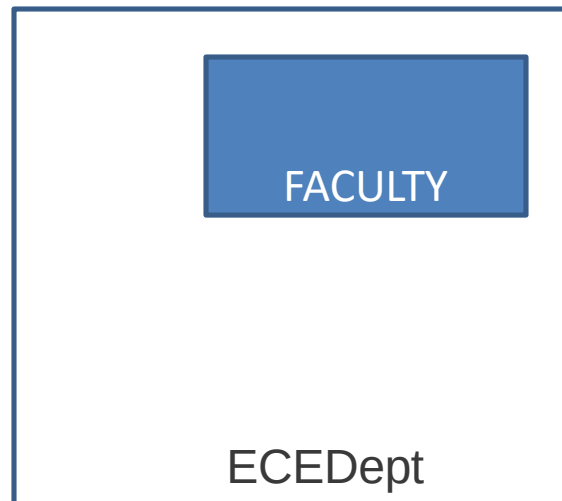
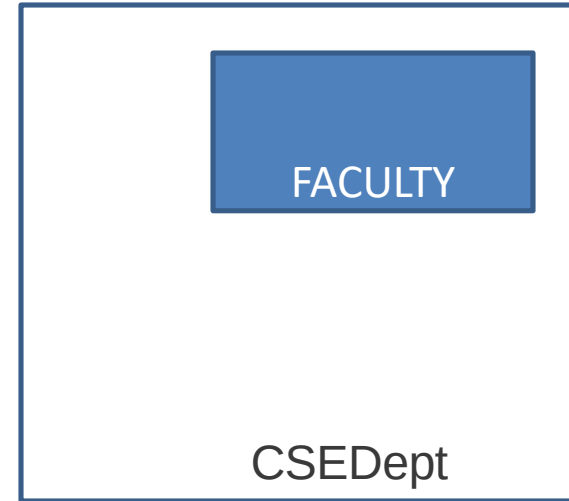
```
class Faculty
{
    public void JP() {
        System.out.println("Java programming...");
    }
    void CP() {
        System.out.println("C Programming...");
    }
    void CPP() {
        System.out.println("C ++ Programming...");
    }
}
```

```
// Inherits feature of Faculty class
class CSEDept extends Faculty{
    public void learn() {
        System.out.println("Computer Science Department ...");
    }
}

// Inherits feature of same Faculty class
class ECEDept extends Faculty{
    public void learn() {
        System.out.println("\nECE Department...");
    }
}

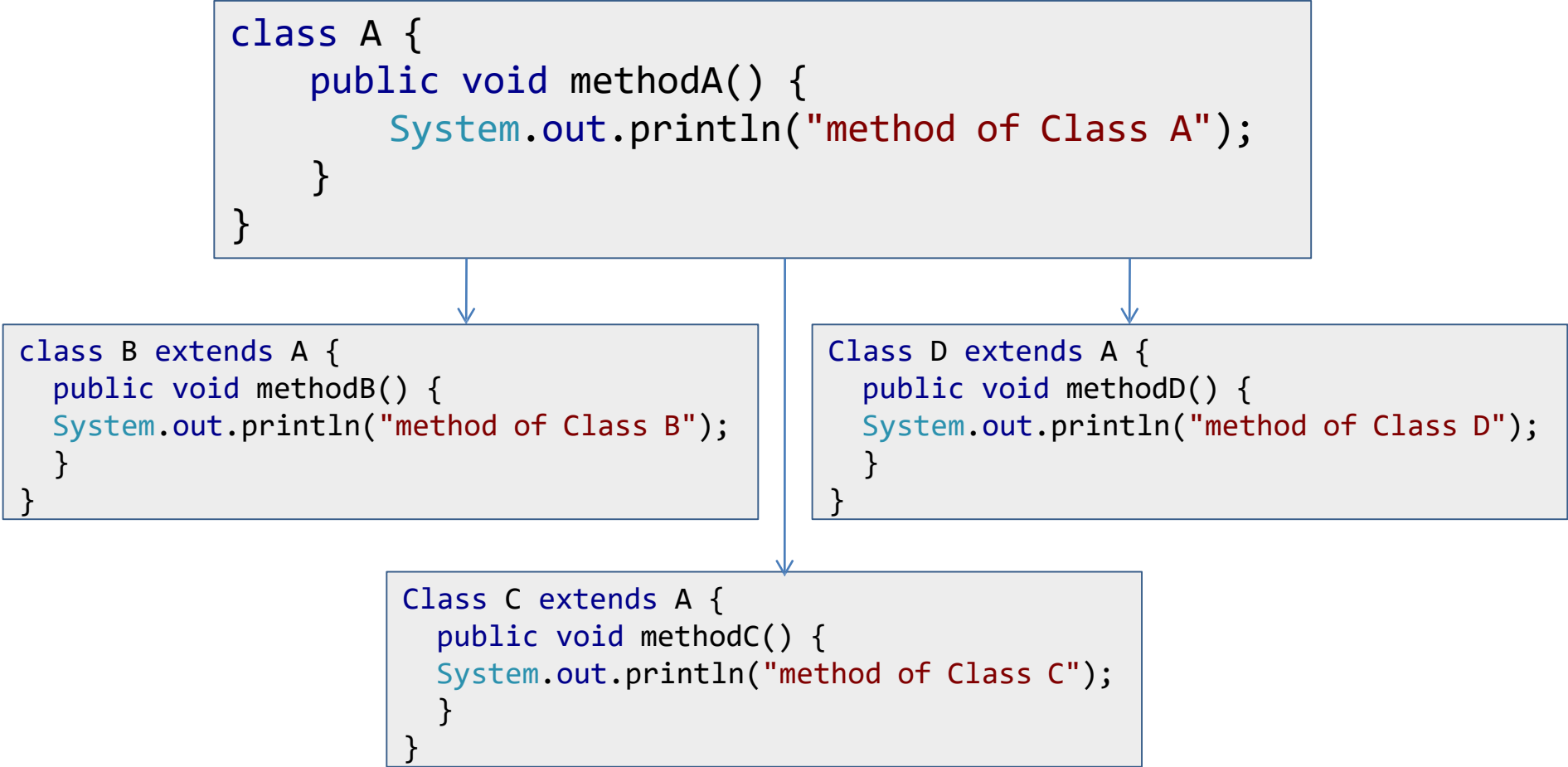
// Inherits feature of same Faculty class
class HSDept extends Faculty{
    public void learn() {
        System.out.println("\nHS Department...");
    }
}
```

```
public class HierarchicalInheritance {  
    public static void main(String[] args) {  
        CSEDept c = new CSEDept();  
        c.learn();  
        c.JP();  
  
        ECEDept e = new ECEDept();  
        e.learn();  
        e.CP();  
  
        HSDept h = new HSDept();  
        h.learn();  
        h.CPP();  
    }  
}
```



Example - 2

```
class A {  
    public void methodA() {  
        System.out.println("method of Class A");  
    }  
}
```



```
class B extends A {  
    public void methodB() {  
        System.out.println("method of Class B");  
    }  
}
```

```
Class D extends A {  
    public void methodD() {  
        System.out.println("method of Class D");  
    }  
}
```

```
Class C extends A {  
    public void methodC() {  
        System.out.println("method of Class C");  
    }  
}
```

```
class JavaExample {  
public static void main(String args[]) {  
    B obj1 = new B();  
    C obj2 = new C();  
    D obj3 = new D(); //All classes can access the method of class A  
  
    obj1.methodA();  
    obj1.methodB();  
  
    obj2.methodA();  
    obj2.methodC();  
  
    obj3.methodA(); }  
    obj3.methodD();  
}
```

private -

Inheritance

- Although a subclass includes all of the members of its superclass, it **cannot access** those members of the superclass that have been declared as **private**.
- *A class member that has been declared as private will remain private to its class. It is not accessible by any code outside its class, including subclasses.*

```
class A {  
    int i; // public by default  
    private int j; // private to A  
    void setij(int x, int y) {  
        i = x;  
        j = y;  
    }  
}  
  
// A's j is not accessible here.  
class B extends A {  
    int total;  
    void sum() {  
        total = i + j; // ERROR, j is not accessible here  
    }  
}
```


A Superclass Variable
Can Reference a
Subclass Object

- A reference variable of a superclass can be assigned a reference to any subclass derived from that superclass.

```
class Data {  
    int data1;  
    int data2;  
}  
class NewData extends Data{  
    int data3;  
    int data4;  
}  
public class Javaapp {  
    public static void main(String[] args) {  
        Data obj = new NewData();  
        obj.data1 = 50;  
        obj.data2 = 100;  
        System.out.println("obj.data1 = "+obj.data1);  
        System.out.println("obj.data2 = "+obj.data2);  
    }  
}
```

Using super

- Whenever a subclass needs to refer to its immediate superclass, it can do so by use of the keyword super.
- super has two general forms.
 - The **first** calls the superclass' constructor.
 - The **second** is used to access a member of the superclass that has been hidden by a member of a subclass.

1. Using super to Call Superclass Constructors

- A subclass can call a constructor defined by its superclass by use of the following form of **super**:
super(arg-list);
- Here, *arg-list* specifies any arguments needed by the constructor in the superclass.
- **super()** must always be the first statement executed inside a subclass' constructor.
- When a subclass calls **super()**, it is calling the constructor of its immediate superclass.

```
// A complete implementation of BoxWeight.
class Box {
private double width;
private double height;
private double depth;
// construct clone of an object
Box(Box ob) { // pass object to constructor
width = ob.width;
height = ob.height;
depth = ob.depth;
}
// constructor used when all dimensions specified
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
// constructor used when no dimensions specified
Box() {
width = -1; // use -1 to indicate
height = -1; // an uninitialized
depth = -1; // box
}
```

```
// constructor used when cube is created
Box(double len) {
width = height = depth = len;
}
// compute and return volume
double volume() {
return width * height * depth;
}
}
```

// BoxWeight now fully implements all constructors.

```
class BoxWeight extends Box {  
    double weight;           // weight of box  
    // construct clone of an object  
    BoxWeight(BoxWeight ob) { // pass object to constructor  
        super(ob);  
        weight = ob.weight;  
    }  
    // constructor when all parameters are specified  
    BoxWeight(double w, double h, double d, double m) {  
        super(w, h, d);      // call superclass constructor  
        weight = m;  
    }  
    // default constructor  
    BoxWeight() {  
        super();  
        weight = -1;  
    }  
    // constructor used when cube is created  
    BoxWeight(double len, double m) {  
        super(len);  
        weight = m;  
    }  
}
```

```

class DemoSuper {
    public static void main(String args[]) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
        BoxWeight mybox3 = new BoxWeight();
        BoxWeight mycube = new BoxWeight(3, 2);
        BoxWeight myclone = new BoxWeight(mybox1);
        double vol;
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        System.out.println("Weight of mybox1 is " + mybox1.weight);
        System.out.println();
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        System.out.println("Weight of mybox2 is " + mybox2.weight);
        System.out.println();
        vol = mybox3.volume();
        System.out.println("Volume of mybox3 is " + vol);
        System.out.println("Weight of mybox3 is " + mybox3.weight);
        System.out.println();
        vol = myclone.volume();
        System.out.println("Volume of myclone is " + vol);
        System.out.println("Weight of myclone is " + myclone.weight);
        System.out.println();
        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
        System.out.println("Weight of mycube is " + mycube.weight);
        System.out.println();
    }
}

```

OUTPUT

```

Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3
Volume of mybox2 is 24.0
Weight of mybox2 is 0.076
Volume of mybox3 is -1.0
Weight of mybox3 is -1.0
Volume of myclone is 3000.0
Weight of myclone is 34.3
Volume of mycube is 27.0
Weight of mycube is 2.0

```

2. A Second Use for super

- The second form of super acts somewhat like **this**, except that it always refers to the superclass of the subclass in which it is used.

super.member

- Here, *member* can be either a method or an instance variable.
- This second form of super is most applicable to situations in which member names of a **subclass** **hide members** by the same name in the superclass.

// Using super to overcome name hiding.

```
class A {  
    int i;  
}  
class B extends A {  
    int i;          // this i hides the i in A  
    B(int a, int b) {  
        super.i = a;    // i in A  
        i = b;          // i in B  
    }  
    void show() {  
        System.out.println("i in superclass: " + super.i);  
        System.out.println("i in subclass: " + i);  
    }  
}  
class UseSuper {  
    public static void main(String args[]) {  
        B subOb = new B(1, 2);  
        subOb.show();  
    }  
}
```

OUTPUT

i in superclass: 1

i in subclass: 2

When Constructors

Are Called

- When a class hierarchy is created, in what order are the constructors for the classes that make up the hierarchy called?
- The answer is that in a class hierarchy, constructors are called in **order of derivation**, from **superclass to subclass**.
- Further, since `super( )` must be the first statement executed in a subclass' constructor, this order is the same whether or not `super( )` is used.
- If `super( )` is **not used**, then the default or parameterless constructor of each superclass will be executed.

```
// Create a super class.
class A {
A() {
    System.out.println("Inside A's constructor.");
}
}

// Create a subclass by extending class A.
class B extends A {
B() {
    System.out.println("Inside B's constructor.");
}
}

// Create another subclass by extending B.
class C extends B {
C() {
    System.out.println("Inside C's constructor.");
}
}

class CallingCons {
public static void main(String args[]) {
C c = new C();
}
}
```

OUTPUT

```
Inside A's constructor
Inside B's constructor
Inside C's constructor
```

Method Overriding

- In a class hierarchy, when a method in a subclass has the **same name and type signature** as a method in its superclass, then the method in the subclass is said to *override the method in* the superclass.
- When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass.
- The version of the method defined by the superclass will be **hidden**.

// Method overriding.

```
class A {  
    int i, j;  
    A(int a, int b) {  
        i = a;  
        j = b;  
    }  
    // display i and j  
    void show() {  
        System.out.println("i and j: " + i + " " + j);  
    }  
}  
class B extends A {  
    int k;  
    B(int a, int b, int c) {  
        super(a, b);  
        k = c;  
    }  
    // display k – this overrides show() in A  
    void show() {  
        System.out.println("k: " + k);  
    }  
}
```

```
class Override {  
    public static void main(String args[]) {  
        B subOb = new B(1, 2, 3);  
        subOb.show(); // this calls show() in B  
    }  
}
```

OUTPUT
k: 3

If you wish to access the superclass version of an overridden method,

```
class B extends A {  
    int k;  
    B(int a, int b, int c) {  
        super(a, b);  
        k = c;  
    }  
    void show() {  
        super.show(); // this calls A's show()  
        System.out.println("k: " + k);  
    }  
}
```

Polymorphism in Java

- **Polymorphism in Java** is a concept by which we can perform a *single action in different ways*.
- Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.
- There are **two** types of polymorphism in Java:
 - compile-time polymorphism (method overloading) and
 - runtime polymorphism (method overriding).

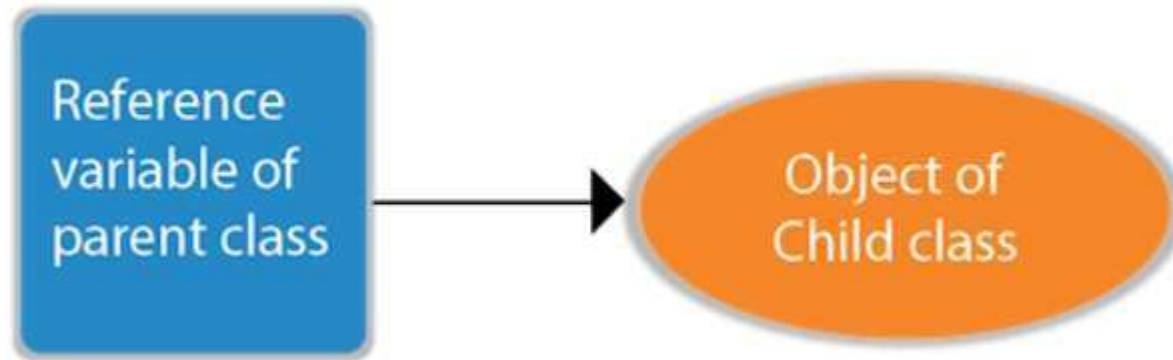
Runtime polymorphism

- Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.
- Dynamic method dispatch is important because this is how Java implements run-time polymorph
- In this process, an **overridden** method is called through the **reference variable of a superclass**.
- The determination of the method to be called is based on the object being referred to by the reference variable.
- If the reference variable of Parent class refers to the object of Child class, it is known as **upcasting**.
- At run-time, it depends on the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.

- In Java, we can override methods only, **not the variables(data members)**, so runtime polymorphism **cannot** be achieved by data members.
- If the reference variable of Parent class refers to the object of Child class, it is known as **upcasting**.

```
class A{  
}  
class B extends A{  
}
```

```
A a=new B();    //upcasting
```



// A Java program to illustrate Dynamic Method Dispatch using hierarchical inheritance

class A

{

void m1()

{

System.out.println("Inside A's m1 method");

}

}

class B extends A

{

// overriding m1()

void m1()

{

System.out.println("Inside B's m1 method");

}

}

class C extends A

{

// overriding m1()

void m1()

{

System.out.println("Inside C's m1 method");

}

}

```
// Driver class
class Dispatch
{
    public static void main(String args[])
    {
        // object of type A
        A a = new A();
        // object of type B
        B b = new B();
        // object of type C
        C c = new C();
        // obtain a reference of type A
        A ref;

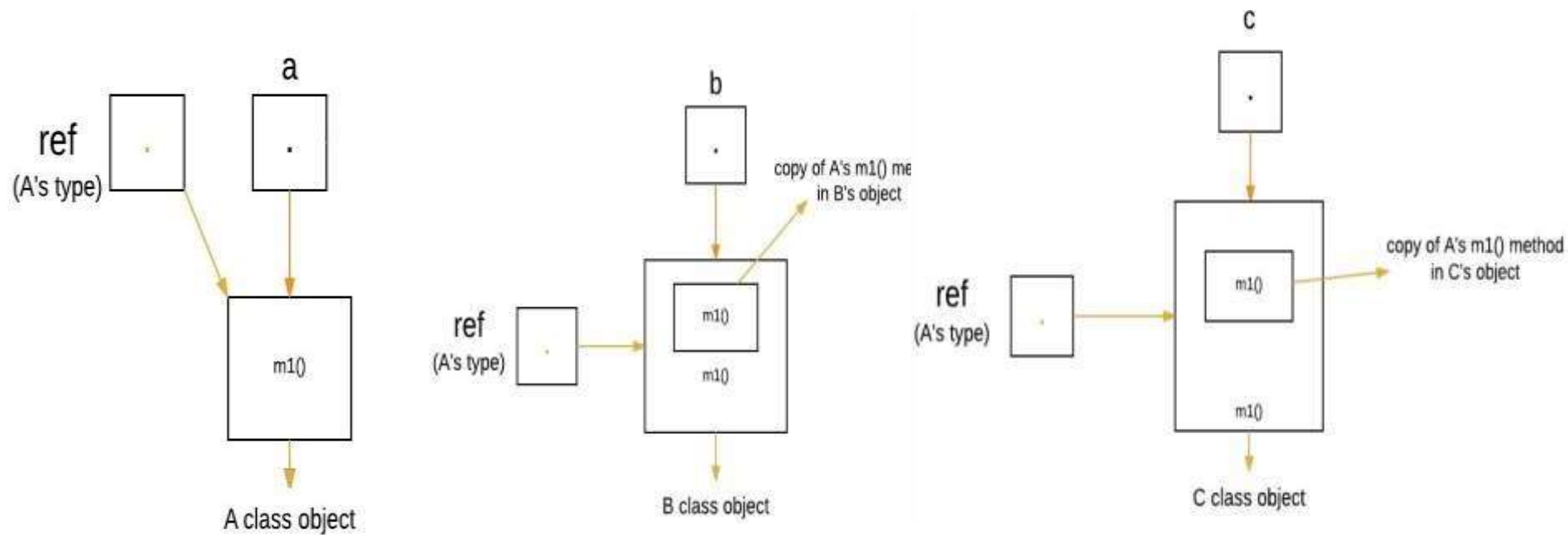
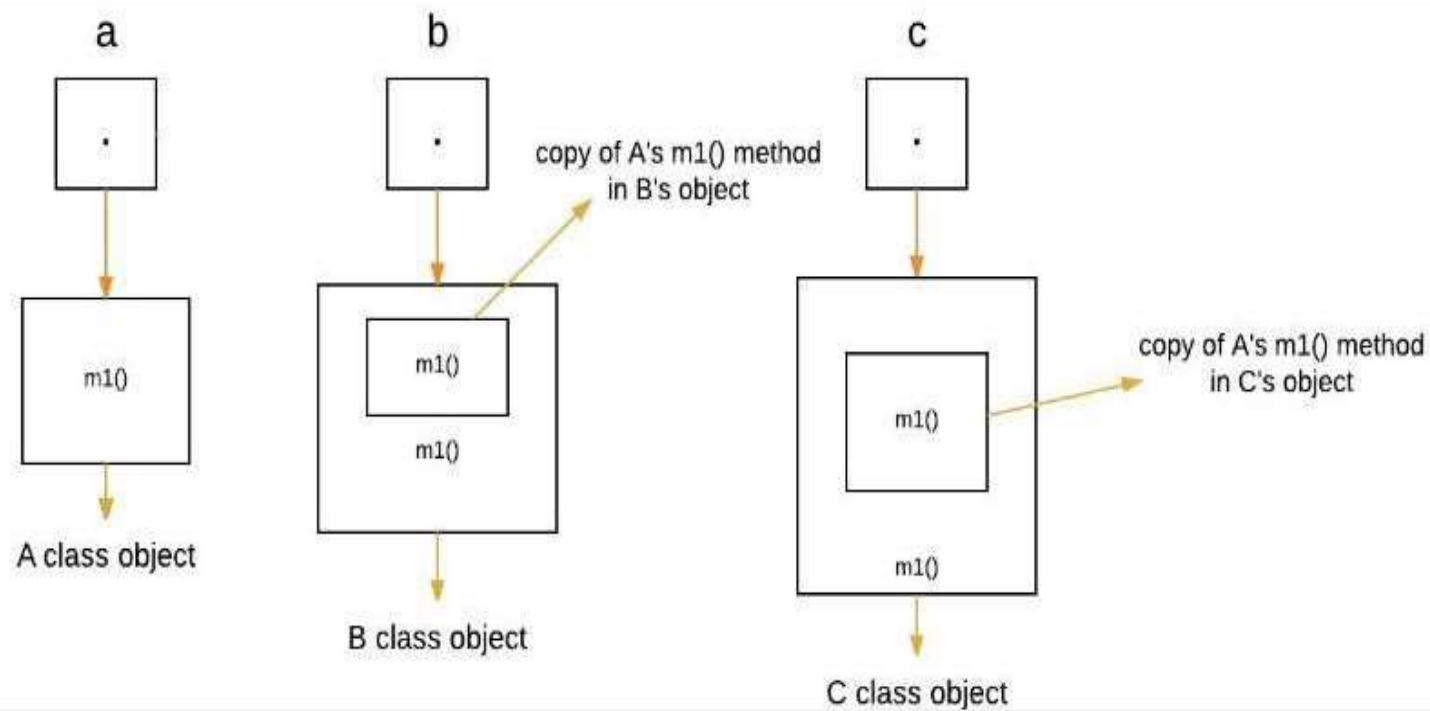
        ref = a;           // ref refers to an A object
        ref.m1();           // calling A's version of m1()

        ref = b;           // now ref refers to a B object
        ref.m1();           // calling B's version of m1()

        ref = c;           // now ref refers to a C object
        ref.m1();           // calling C's version of m1()
    }
}
```

Output:

```
Inside A's m1 method
Inside B's m1 method
Inside C's m1 method
```



Static vs Dynamic binding

- Static binding is done during compile-time while dynamic binding is done during run-time.
- private, final and static methods and variables uses static binding and bonded by compiler while overridden methods are bonded during runtime based upon type of runtime object

Why Overridden

Methods?

- Overridden methods allow Java to support run-time polymorphism.
- Polymorphism is essential to object-oriented programming for one reason: it allows a general class to specify methods that will be common to all of its derivatives, while allowing subclasses to define the specific implementation of some or all of those methods.
- Overridden methods are another way that Java implements the “one interface, multiple methods” aspect of polymorphism.
- Dynamic, run-time polymorphism is one of the most powerful mechanisms that objectoriented design brings to bear on code reuse and robustness.
- *C++ or C# will recognize that overridden methods in Java are similar to **virtual functions** in those languages.*

Abstract Classes

- A class which is declared with the abstract keyword is known as an abstract class in Java. It can have abstract and non-abstract methods (method with the body).
- **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.
- Another way, it shows only essential things to the user and hides the internal details, for **example**, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.
- Abstraction lets you focus on what the object does instead of how it does it.
- There are two ways to achieve abstraction in java
 - Abstract class (0 to 100%)
 - Interface (100%)

```
abstract class class-name
{

    non-abstract/regular methods    //defined within { }
    {
    }

    abstract methods();    //ending with a semicolon;
}
```

An abstract class may contain a mix of non-abstract methods and abstract methods, or only abstract methods ,or only non-abstract regular methods.

Rules for Java Abstract class



1

An abstract class must be declared with an abstract keyword.

2

It can have abstract and non-abstract methods.

3

It cannot be instantiated.

4

It can have final methods

5

It can have constructors and static methods also.

- An abstract class must be declared with an **abstract** keyword.
- It can have abstract and non-abstract methods.
- It **cannot** be instantiated. An abstract class cannot be directly instantiated with the new operator. Such objects would be useless, because an abstract class is not fully defined.
- It can have constructors and static methods also.
- Cannot declare abstract constructors, or abstract static methods.
- It can have final methods which will force the subclass not to change the body of the method.
- If there is an abstract method in a class, that class **must be abstract**.
- If you are extending an abstract class that has an abstract method, you must either provide the implementation of the method or make this class abstract.

Abstract Method in Java

- A method which is declared as abstract and **does not have implementation** is known as an abstract method.

abstract type name(parameter-list);

abstract void printStatus();

```
abstract class Bank{  
    abstract int  
    getRateOfInterest();  
}
```

```
class SBI extends Bank{  
    int getRateOfInterest(){  
        return 7;}  
}
```

```
class PNB extends Bank{  
    int getRateOfInterest(){  
        return 8;}  
}
```

```
class TestBank{  
    public static void main(String args[]){  
        Bank b;  
        b=new SBI();  
        System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");  
  
        b=new PNB();  
        System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");  
  
    }  
}
```

OUTPUT

Rate of Interest is: 7 %

Rate of Interest is: 8 %

//Example of an abstract class that has abstract and non-abstract methods

```
abstract class Bike{  
    Bike(){  
        System.out.println("bike is created");}  
    abstract void run();  
    void changeGear(){  
        System.out.println("gear changed");}  
}
```

//Creating a Child class which inherits Abstract class

```
class Honda extends Bike{  
    void run(){  
        System.out.println("running safely..");}  
}
```

//Creating a Test class which calls abstract and non-abstract methods

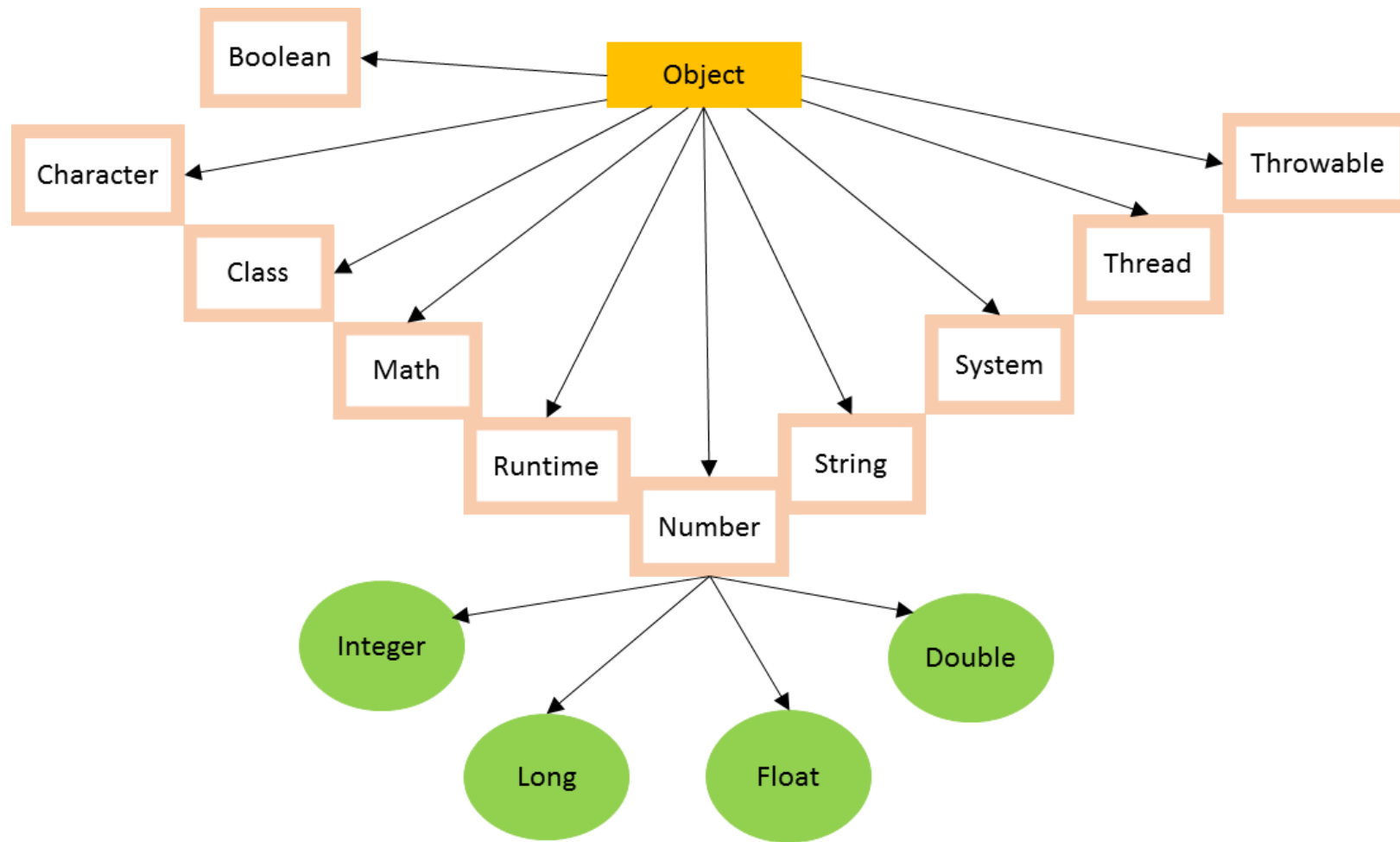
```
class TestAbstraction2{  
    public static void main(String args[]){  
        Bike obj = new Honda();  
        obj.run();  
        obj.changeGear();  
    }  
}
```

OUTPUT

bike is created
running safely..
gear changed

The Object Class

- There is one special class, **Object**, defined by Java.
- All other classes are **subclasses** of Object. That is, Object is a superclass of all other classes. This means that a reference variable of type Object can refer to an object of any other class.
- Also, since arrays are implemented as classes, a variable of type Object can also refer to any array.



Method	Purpose
Object clone()	Creates a new object that is the same as the object being cloned.
boolean equals(Object <i>object</i>)	Determines whether one object is equal to another.
void finalize()	Called before an unused object is recycled.
Class getClass()	Obtains the class of an object at run time.
int hashCode()	Returns the hash code associated with the invoking object.
void notify()	Resumes execution of a thread waiting on the invoking object.
void notifyAll()	Resumes execution of all threads waiting on the invoking object.
String toString()	Returns a string that describes the object.
void wait() void wait(long <i>milliseconds</i>) void wait(long <i>milliseconds</i> , int <i>nanoseconds</i>)	Waits on another thread of execution.